Capability	LangGraph	CrewAI	Microsoft AF	Solace Agent Mesh	You Build
Workflow orchestration	✓	✓	✓	✓	-
State persistence	✓ native	◐ opaque	◐ limited	✓ event-driven	-
Human-in-the-Loop	✓	◐ basic	✓	✓	-
Distributed trace/Audit	◐ LangSmith	✗	◐ Azure only	✓ built-in	✓
Agent IAM/ Least-privilege	✗	✗	◐ Azure-bound	✓ SSO+RBAC	✓
Multi-agent trust boundary	✗	✗	✗	✓ gateway	✓
Cost/resource efficiency and governance	◐ manual	✗	◐ Azure Cost	◐ data management	✓
Compliance/ Data residency	✗	✗	◐ Azure platform	✓ SOC2·ISO27001	✓

Table 7: Framework coverage vs enterprise requirements

All of these are genuine capabilities, not marketing. But a precise reading of each reveals where the framework layer ends and the infrastructure layer begins, and which approach leaves you to assemble that infrastructure yourself.

Table 7 is not a criticism of the orchestration frameworks. They were all designed to solve the hard problem of orchestration: how to define, sequence, and execute agent behaviour. They solve that well. But they were not designed to provide a production security model, a cross-layer audit trail, agent-scoped IAM, multi-agent trust validation, or workflow-level cost governance. Those requirements sit above the framework layer. If you want them, you assemble them yourself -- LangSmith or Langfuse for tracing, a secrets manager for credentials, a policy engine for access control, an event broker for reliable delivery. Each one is a separate project.

Solace Agent Mesh starts from a different place entirely. It treats the enterprise infrastructure requirements as the foundation, not the finishing touch. Because every agent interaction flows through the event broker, observability and fault tolerance are built in rather than bolted on. Because security sits in the gateway layer by default, it doesn't require a separate identity integration project. The trade-off is real: Agent Mesh is less composable for teams that want to wire up arbitrary orchestration patterns from scratch, and it introduces a dependency on Solace's event broker infrastructure. But for enterprises moving from proof-of-concept to govern production, that trade-off is often exactly what's needed.

Think of the framework as the engine. The three pillars -- operational integrity, security and trust, governance at scale -- are the chassis, the brakes, and the safety systems. Shipping an engine without the rest of the vehicle is what produces the 40% cancellation rate Gartner is projecting. The real question isn't which framework to pick. It's whether the infrastructure layer that production demands gets assembled piecemeal over time or is built in from the start.

The hardest part isn't the build

None of the failure modes, trust gaps, or governance deficits described here are exotic. They're the standard challenges of distributed systems, applied to a new class of actor that reasons instead of just executes. We've solved versions of these problems before — in micro-services, in cloud-native architectures, in event-driven integration. The patterns exist. They just haven't been applied to agents yet.

The teams that treat these as architectural requirements from day one, rather than operational problems to fix after launch, are the ones whose agentic systems survive contact with production. The others will have great demos. You built the agent — now build the infrastructure around it.

END

By: Giri Venkatesan

The author is principal developer advocate & systems architect at Solace, working at the intersection of agentic AI, event-driven architecture, and enterprise integration. He has spent over two decades building distributed systems across integration, master data management, and cloud-native applications.